

Fiches DevSecOps — recettes réutilisables

Parties 1 à 7 du parcours. Chaque fiche est écrite pour être appliquée à n'importe quel projet, pas seulement à celui-ci.

Sommaire

	Fiche	Ce qu'on y trouve
0	Principes transversaux	les sept lois qui reviennent partout
1	Architecture d'un service	package-by-feature, domaine isolé, invariants
2	Base de données	Flyway + JPA, transactions, concurrence, erreurs HTTP
3	Tests	où placer quoi, la recette en 5 temps, les pièges
4	Testcontainers	singleton, reuse, état initial, le piège <code>@Transactional</code>
5	Dockerfile de production	multi-stage, non-root, healthcheck, <code>.dockerignore</code>
6	Pipeline GitHub Actions	structure, moindre privilège, épinglage, cache
7	Gitleaks	règles et entropie, <code>fetch-depth</code> , prouver que ça marche
—	Annexe	commandes de référence

Comment lire ces fiches — chacune suit le même plan : *quand l'appliquer, la recette, les pièges* (souvent des échecs silencieux), *comment vérifier que ça fonctionne vraiment*. Les tableaux « pièges » sont la partie la plus rentable : ce sont des heures perdues déjà payées.

0 · Les principes transversaux

Sept lois rencontrées dans plusieurs parties. Elles reviennent partout ; les connaître évite de réapprendre la même chose sous un autre déguisement.

1. Un nom est une promesse, une empreinte est un fait

Un tag, une version, une branche : mutables. Quelqu'un peut les faire pointer ailleurs sans que ton fichier change d'un caractère.

Objet	Nom (mutable)	Empreinte (immuable)
Image Docker	postgres:18-alpine	@sha256:1ff763...
Action GitHub	actions/checkout@v5	@fbc6f399...
Image produite	mon-app:0.0.1	mon-app:\${{ github.sha }}

Contrepartie : une chose épinglée ne reçoit plus les correctifs. Il faut un mécanisme de mise à jour délibéré — Dependabot, Renovate, ratchet. *Épingler sans mécanisme de mise à jour, c'est échanger un risque contre un autre.*

2. On masque, on n'efface jamais

	Ce que « supprimer » fait réellement
Couche Docker	pose un marqueur ; le fichier reste dans la couche, extractible
Historique Git	retire du présent ; le commit précédent garde tout
Migration Flyway appliquée	le checksum est enregistré ; la modifier casse le démarrage

La seule protection est de **ne jamais faire entrer** ce qui ne doit pas y être. `.dockerignore`, `.gitignore`, et une vérification avant le premier commit.

3. Une option mal orthographiée est silencieusement ignorée

```
-DskipTests      Maven crée une propriété que personne ne lit
type: gha        entrée inconnue d'une action ignorée
fetch-depth: 1   (défaut) un seul commit rien à scanner
```

Rien ne casse. Tout est vert. La vérification n'a pas eu lieu. C'est le mode d'échec le plus dangereux d'un pipeline.

4. Un contrôle qui n'a jamais rien trouvé n'est pas un contrôle

Un outil de sécurité doit être **prouvé** : on lui donne volontairement ce qu'il doit détecter, on vérifie qu'il crie, on nettoie. Sans cette preuve, on ne sait pas si le dépôt est sain ou si l'outil est cassé.

Corollaire : un job **rouge** ne prouve rien non plus — il peut échouer *avant* d'avoir vérifié quoi que ce soit. La seule preuve est le rapport qui nomme la règle, le fichier et la ligne.

5. Qui détient la vérité ?

Avant d'écrire un test, demande : *la vérité de ce comportement est-elle dans mon code, ou dans l'infrastructure ?*

- Code Java → mock, quelques millisecondes
- PostgreSQL, Hibernate, la couche HTTP → il faut la vraie chose

Monter en base « pour être sûr » quand la vérité est dans ton code, c'est comment on obtient une suite de quatre minutes que plus personne ne lance.

6. Moindre privilège, déclaré au plus près

`permissions: contents: read` au niveau du workflow, et on élargit **sur le job qui en a besoin**, ligne par ligne. Une permission déclarée sur un job ne fuit pas vers les autres.

7. Fail-fast, et un échec bruyant vaut mieux qu'un succès silencieux

Une application qui refuse de démarrer sans sa configuration est meilleure qu'une application qui démarre et renvoie des 500. `ddl-auto=validate`, un `Exited (1)` faute de `DB_URL`, un pipeline qui s'arrête au premier contrôle : c'est le même principe.

Fiche 1 · Architecture d'un service

Quand : au démarrage d'un projet, avant la première ligne de code métier.

La recette

1. **Organiser par fonctionnalité**, pas par couche technique. `order/` plutôt que `controllers/`, `services/`, `repositories/`. On lit un dossier et on comprend le métier.

2. **Hexagonal pragmatique** : appliqué au seul agrégat qui porte des règles. Pas partout — le reste serait de la cérémonie.

```
order/ ── domain/ objets métier – ZÉRO import framework ── web/ contrôleur, DTO d'entrée et de sortie, mapper ── persistence/ entités, repositories, mapper ── OrderService orchestration
```

3. **Trois représentations distinctes** : DTO ≠ objet métier ≠ entité. Le domaine ne connaît ni le web ni la persistance.

4. **Les règles métier vivent dans l'objet**, pas dans le service. Une machine à états s'implémente dans l'entité métier, avec une méthode privée `transitionTo()` et des méthodes publiques nommées.

5. **Deux portes d'entrée seulement** vers un objet métier : - un constructeur de **création** (état initial forcé) - une fabrique de **reconstitution** (état restauré depuis la base)

Sans la seconde, un objet rechargé repart à l'état initial et la machine à états devient contournable.

Les pièges

Mass assignment	un DTO d'entrée ne contient que ce que le client a le droit de fixer. Le prix vient du catalogue, jamais de la requête
Setters publics	ils permettent de contourner les invariants. Constructeur validant, getters seuls
Collection non copiée	<code>List.copyOf()</code> : sinon l'appelant garde une référence et modifie l'objet après coup
GET qui change l'état	jamais. Préchargement navigateur, CSRF. Les transitions sont des <code>POST</code>

Vérifier

Un test unitaire pur, sans framework : créer, transiter, vérifier qu'une transition interdite lève, vérifier qu'une reconstitution restaure bien l'état.

Fiche 2 · Base de données — Flyway + JPA

Quand : dès qu'il y a une base, avant la première entité.

La recette

1. Flyway commande le schéma, Hibernate le valide.

```
properties spring.jpa.hibernate.ddl-auto=validate spring.jpa.open-in-view=false
```

`validate` compare les entités au schéma **à chaque démarrage**. Une entité modifiée sans migration fait échouer le boot : fail-fast.

2. Migrations versionnées dans `db/migration`, jamais modifiées une fois appliquées. Une correction = une nouvelle migration.

3. Les données de développement dans un dossier séparé, activé par profil :

```
properties # application-dev.properties
```

```
spring.flyway.locations=classpath:db/migration,classpath:db/dev
```

Le seed ne part jamais en production.

4. Secrets hors du dépôt : `.env` gitignoré, `.env.example` commité,

```
spring.config.import=optional:file:.env[.properties]
```

5. Une transaction par cas d'usage : `@Transactional` sur les méthodes du service, `readOnly = true` sur les lectures.

Les pièges

Modifier une entité gérée	dans une transaction, l'entité chargée est surveillée. On modifie le champ, le <i>dirty checking</i> écrit l'UPDATE. Surtout pas de <code>save()</code> d'une entité reconstruite : <code>created_at</code> à null, lignes détruites par <code>orphanRemoval</code> , UPDATE complet
Association bidirectionnelle	une méthode <code>addLine()</code> sur le parent qui maintient les deux côtés . Sans <code>line.setOrder(this)</code> , la clé étrangère part à NULL
<code>@Enumerated(ORDINAL)</code>	interdit : réordonner l'enum corrompt les données. Toujours <code>STRING</code>
<code>@GeneratedValue</code> + <code>setId()</code>	les deux ne peuvent pas gagner. Choisir
<code>toString()</code> avec une association LAZY	un simple log déclenche une requête ou une <code>LazyInitializationException</code>
Le flush	<code>save()</code> n'écrit rien : l'INSERT part au commit. La stack trace pointe la fin de la transaction, pas la ligne fautive
Index sur les clés étrangères	automatique sur la PK, pas sur les FK. Indispensable si la table parente subit des suppressions (<code>ON DELETE RESTRICT</code>)
<code>BigDecimal</code> pour l'argent	jamais <code>double</code> . Et <code>NUMERIC(19,2)</code> en base

Concurrence

`@Version` sur l'entité + une colonne `version BIGINT NOT NULL DEFAULT 0`. Sans lui, deux requêtes concurrentes lisent le même état, passent toutes deux le contrôle métier, et la seconde écrase la première en silence. Le perdant reçoit un `OptimisticLockingFailureException` → **409**.

Gestion des erreurs

`@RestControllerAdvice` étendant `ResponseEntityExceptionHandler` (pour que les exceptions internes de Spring gardent leur traitement). Règle unique : **détail dans les logs, message générique dans la réponse**.

Situation	Code
ressource inexistante	404
entrée invalide, référence inconnue	400
conflit d'état, concurrence	409
tout le reste	500 générique

Et `server.error.include-stacktrace=never`.

Fiche 3 · Tests — où placer quoi

Quand : avant d'écrire un pipeline. Sans tests fiables, une CI ne fait que livrer des bugs plus vite.

La règle de placement

Qui détient la vérité de ce comportement — mon code, ou l'infrastructure ?

	coût	quantité	ce qu'on y prouve
bout en bout	~10 s	très peu	le tout branché
tranches Spring	~2 s	peu	contrat HTTP, mapping SQL
service + mocks	~50 ms	moyen	l'orchestration
domaine, JUnit pur	~1 ms	beaucoup	les règles métier

Descends le test au niveau le plus bas qui puisse encore l'attraper.

La recette d'un test, en 5 temps

1. UNE phrase	ce que le test prouve
2. TRACE le chemin	quels appels externes, sur CE chemin précis
3. ARRANGE	when(...) sur ceux-là, et rien d'autre
4. ACT	la méthode testée
5. ASSERT	ce qui doit arriver + verify (never()) pour ce qui ne doit pas

L'étape 2 est la seule qui demande de réfléchir. Si le code sort par une exception à la ligne 1, les appels des lignes suivantes n'ont jamais lieu : les préparer produit une `UnnecessaryStubbingException`.

Les outils

<code>@Mock</code> / <code>@InjectMocks</code>	faux objets + le vrai objet testé
<code>thenReturn</code>	réponse fixe
<code>thenAnswer(a -> a.getArgument(0))</code>	rend ce qu'on lui donne — quand l'objet n'existe pas encore au moment du ARRANGE
<code>thenThrow</code>	tester le chemin d'erreur
<code>verify(mock, never())</code>	prouver qu'une chose n'a pas eu lieu
<code>@WebMvcTest</code>	tranche web : contrôleurs et <code>@RestControllerAdvice</code> , jamais les services
<code>@MockitoBean</code>	<code>@Mock</code> placé dans le contexte Spring (<code>@MockBean</code> est déprécié)
<code>MockMvc</code> + <code>jsonPath</code>	requêtes HTTP sans serveur

Réponses par défaut d'un mock non configuré : `null`, liste vide, `Optional.empty()`, `0`, `false`. Ça marche — mais on écrit quand même le `when` : un test doit *dire* ce qu'il suppose.

Les pièges

<code>Optional.empty() != null</code>	<code>thenReturn(null)</code> sur une méthode qui rend un <code>Optional</code> → NPE, pas l'exception attendue
<code>isEqualTo</code> sur <code>BigDecimal</code>	compare aussi le <i>scale</i> : <code>224.80 != 224.8</code> . Utiliser <code>isEqualByComparingTo</code>
<code>int</code> primitif	ne distingue pas <i>absent</i> de <i>zéro</i> . <code>Integer</code> + <code>@NotNull</code> si la nuance compte
Validation en cascade	<code>@Valid</code> sur la liste, sinon les contraintes des éléments ne sont jamais évaluées
<code>@Positive</code> sur une <code>List</code>	silencieusement inopérant. C'est <code>@NotEmpty</code>
<code>Import .shaded.</code>	jamais. Ce sont des classes privées par accident

Les erreurs de conception

1. Tester les getters pour gonfler la couverture — métrique de vanité
2. Un test sans assertion réelle : le casser volontairement pour vérifier
3. Mocker ce qu'on ne possède pas (`EntityManager` = réimplémenter Hibernate)
4. `@SpringBootTest` partout → suite de 4 minutes → plus personne ne la lance
5. Tests couplés par un état partagé → verts en local, rouges en CI parallèle
6. `verify()` sur tout → on teste l'implémentation, plus le comportement

Vérifier que la suite vaut quelque chose

Casse **une** ligne de production, lance les tests, note lequel tombe. Si aucun ne tombe, tu viens de trouver un trou. C'est plus instructif que n'importe quel pourcentage de couverture.

Fiche 4 · Testcontainers

Quand : dès qu'il faut prouver un comportement détenu par la base — persistance réelle, contraintes, transactions.

Pourquoi pas une base en mémoire

Une base légère embarquée n'est pas ta base. Types différents, syntaxe partielle, et surtout **deux jeux de migrations à maintenir** : le schéma que tu testes cesse d'être le schéma que tu déploies. Son mode compatibilité est le plus dangereux : il marche assez pour endormir, et lâche sur le cas tordu.

La recette

Une classe de base abstraite, dont héritent tous les tests d'intégration :

```
@SpringBootTest
@Sql("/db/reset.sql") // état initial DÉCLARÉ, pas supposé
public abstract class AbstractIntegrationTest {

    @ServiceConnection // Boot injecte URL/user/password
    protected static final PostgreSQLContainer POSTGRES =
        new PostgreSQLContainer("postgres:18-alpine"); // MÊME image qu'en prod

    static {
        POSTGRES.withReuse(true);
        POSTGRES.start(); // jamais arrêté : Ryuk nettoie
    }
}
```

Les deux axes de performance — ils ne s'opposent pas

SINGLETON  combien de conteneurs PENDANT un build
REUSE  le conteneur survit-il ENTRE deux builds

	Pendant un build	Après
<code>@Container</code> par classe	N démarrages, N contextes Spring	détruit
Singleton	1 démarrage, 1 contexte Spring réutilisé	détruit
Singleton + reuse	0 si le conteneur est déjà là	il reste vivant

Le singleton est l'essentiel : il marche partout, y compris en CI, sans configuration. Son gain caché est le plus gros — l'URL JDBC ne changeant plus, Spring réutilise le même contexte applicatif d'une classe à l'autre.

Le reuse est un confort local. Il s'active dans `~/.testcontainers.properties` (`testcontainers.reuse.enable=true`) — **hors du dépôt par construction** : la bibliothèque refuse de lire ce réglage depuis le classpath. La CI repart donc toujours d'une base vierge.

Les pièges

<code>@Transactional</code> sur le test	tout se passe dans une seule transaction, donc un seul cache Hibernate : on relit sa propre modification en mémoire. C'est le piège du mock, avec une vraie base derrière. Pour tester la persistance, la donnée doit sortir de la mémoire et y revenir
État hérité du build précédent	avec <code>reuse</code> , la base garde son contenu. D'où <code>@Sql</code> qui remet l'état initial avant chaque test
Migration modifiée + conteneur réutilisé	Flyway refuse (checksum). Injecter une empreinte des fichiers SQL dans un label du conteneur force un conteneur neuf automatiquement
Nombre magique dans une assertion	il doit venir d'une source qu'on peut montrer du doigt — le script <code>@Sql</code> , pas la mémoire

Autres conteneurs

- `GenericContainer` quand il n'existe pas de classe dédiée. **La stratégie d'attente est le point critique** : sans elle, le test démarre avant que le service ne réponde → un test instable, pire qu'un test absent.
- `Network` + `withNetworkAliases("db")` quand deux conteneurs doivent se parler. Depuis l'intérieur d'un conteneur, `localhost` désigne ce conteneur-là.

Fiche 5 · Dockerfile de production

Quand : avant de publier quoi que ce soit.

La recette

```
# ----- étape 1 : builder (jetée) -----
FROM eclipse-temurin:21-jdk-alpine@sha256:... AS builder
WORKDIR /app

# ce qui change rarement d'abord
COPY pom.xml mvnw ./
COPY .mvn .mvn
RUN chmod +x mvnw && ./mvnw -B dependency:go-offline

# ce qui change à chaque commit ensuite
COPY src src
RUN ./mvnw -B package -DskipTests && mv target/*.jar /app/app.jar

# ----- étape 2 : runtime (publiée) -----
FROM eclipse-temurin:21-jre-alpine@sha256:...

LABEL org.opencontainers.image.title="mon-app" \
      org.opencontainers.image.source="https://github.com/..."

RUN addgroup -S -g 10001 app && adduser -S -u 10001 -G app app
WORKDIR /app
COPY --from=builder --chown=app:app /app/app.jar app.jar
USER 10001:10001
EXPOSE 8080

HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health | grep -q '"status":"UP"' || exit 1

ENTRYPOINT ["java", "-jar", "/app/app.jar"]
```

`.dockerignore` — en liste blanche

```
*
!pom.xml
!mvnw
!.mvn
!src

**/.DS_Store
```

On refuse tout, on réautorise l'indispensable. Un nouveau fichier de secrets à la racine est exclu d'office. En liste noire, il entrerait. **La dernière règle qui correspond l'emporte** — attention aux exclusions placées après

les réautorisation.

Docker ne lit **pas** `.gitignore`. Deux outils, deux fichiers.

Vérifier ce qui entre dans le contexte

```
docker build --no-cache --progress=plain -f - - <<'EOF'
FROM alpine
COPY . /ctx
RUN find /ctx | sort
EOF
```

Regarde aussi la ligne `transferring context: ... kB`. Quelques dizaines de Mo signifient que `target/` et `.git` partent au démon à chaque build.

Les points qui font la différence

	Pourquoi
Multi-stage	le JDK, Maven, le cache <code>.m2</code> et les sources ne partent jamais en production. Ce n'est pas une question de poids mais de surface d'attaque : un attaquant y trouverait un atelier complet
Ordre des <code>COPY</code>	ce qui change rarement en premier. Le téléchargement des dépendances devient une couche à part, réutilisée tant que le <code>pom.xml</code> ne bouge pas
UID numérique	<code>runAsNonRoot: true</code> de Kubernetes ne sait pas résoudre un nom d'utilisateur. Un UID élevé (10001) évite aussi les collisions sur les volumes
Forme exec	la JVM devient le processus 1 et reçoit <code>SIGTERM</code> → arrêt gracieux. En forme shell, <code>/bin/sh</code> est le processus 1 et ne transmet rien
<code>--start-period</code>	une JVM met 5 à 15 s à démarrer. Sans cette fenêtre, les échecs normaux du démarrage comptent dans les <code>retries</code>
Épingleage par digest	un tag bouge, un digest non
Aucune configuration dans l'image	la même image va en dev, en recette et en production. Un <code>Exited (1)</code> faute de configuration est la preuve que l'image ne contient aucun secret

Choisir l'image runtime

	Taille	CVE	Débogage	Note
jre-alpine	~180 Mo	peu	shell présent	musl , pas glibc
distroless/java	~230 Mo	très peu	aucun shell	pas de <code>HEALTHCHECK</code> possible
jre (Ubuntu)	~280 Mo	beaucoup	complet	glibc

Alpine utilise **musl** : les bibliothèques natives compilées pour glibc (`netty-tcnative` , certains JNI) échouent au chargement. Sans dépendance native, aucun risque.

Liveness ≠ readiness

Un endpoint de santé qui inclut la base est correct en **readiness** (« ne m'envoie plus de trafic ») et catastrophique en **liveness** : une panne de base ferait redémarrer tous les pods en boucle, et au retour de la base ils la submergeraient tous ensemble.

Une sonde de liveness répond à « ce processus est-il irrécupérable ? », jamais à « mes dépendances vont-elles bien ? »

Le `HEALTHCHECK` du Dockerfile est **ignoré par Kubernetes** — il sert à Docker et à Compose (`depends_on: condition: service_healthy`).

Fiche 6 · Pipeline GitHub Actions

Quand : dès qu'un dépôt existe et que des tests valent la peine d'être lancés automatiquement.

Le modèle

```
workflow → jobs (DICTIONNAIRE, nommés, isolés) → steps (LISTE ordonnée)
```

Les jobs sont **nommés** parce qu'on les référence (`needs: build`). Les steps sont une liste parce qu'ils s'exécutent dans l'ordre.

Chaque job démarre sur une machine neuve et vide. D'où le `checkout` répété dans chaque job, et `upload-artifact` / `download-artifact` comme seul chemin pour faire passer un fichier de l'un à l'autre.

Le squelette

```
name: CI

on:
  push:
    branches: [main]      # le filtre ne concerne QUE push
  pull_request:           # nu = toutes les PR

permissions:
  contents: read          # le GITHUB_TOKEN peut ÉCRIRE par défaut

concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-22.04
    steps:
      - uses: actions/checkout@<sha> # vX.Y.Z
        with:
          persist-credentials: false
      - uses: actions/setup-java@<sha> # vX.Y.Z
        with:
          distribution: temurin
          java-version: '21'
          cache: maven
      - run: ./mvnw -B verify
      - if: always() # sinon SAUTÉ quand les tests cassent
        uses: actions/upload-artifact@<sha>
        with:
          name: rapports-tests
          path: target/surefire-reports/
          retention-days: 7
```

Sécurité du pipeline lui-même

	Pourquoi
<code>permissions: contents: read</code>	le <code>GITHUB_TOKEN</code> peut pousser des commits et créer des releases par défaut. On élargit sur le job qui en a besoin, jamais globalement
<code>persist-credentials: false</code>	sinon <code>checkout</code> laisse le jeton dans <code>.git/config</code> sur le runner, lisible par toute étape suivante
<code>pull_request</code> , jamais <code>pull_request_target</code>	le second exécute le workflow de la branche de base, avec les secrets et un jeton en écriture , sur du code proposé par n'importe qui
Actions épinglées par SHA	un tag est mutable. En 2025, une action très utilisée a vu ses tags réécrits vers un commit qui déversait les secrets de CI dans les logs
Commentaire de version	sans lui, plus personne — ni toi, ni Dependabot — ne sait quelle version tourne. L'épinglage devient un cul-de-sac
Épingler aussi l'outil	une action peut télécharger un binaire dont elle choisit la version (<code>GITLEAKS_VERSION</code> , etc.)

Sur un événement `pull_request`

GitHub exécute le workflow **de la branche de la PR**, pas celui de `main`. C'est ce qui permet de tester une modification de CI avant de la fusionner — et c'est exactement ce que `pull_request_target` inverse, d'où son danger.

Choisir une action

- L'éditeur d'abord** : `docker/` pour Docker, `actions/` pour GitHub, `aquasecurity/` pour Trivy. L'organisation qui publie l'action doit être celle qui publie l'outil.
- Trente secondes de vérification** : bandeau du README (souvent « deprecated »), date du dernier commit, rythme des releases, « Used by ».
- La question que peu se posent** : *ai-je besoin d'une action ?* Beaucoup ne sont que trois lignes de shell emballées, et chacune est un tiers de plus qui s'exécute dans ta CI. Une action se justifie quand elle apporte une vraie mécanique — cache, authentification, annotations de PR.
- Astuce** : lire les workflows de projets sérieux (`spring-projects/spring-boot`, `testcontainers/...`). Revue de code gratuite.

Récupérer un SHA

```
# avec gh
gh api repos/actions/checkout/commits/v5 --jq .sha

# avec git seul — ^{} déréférence un tag annoté vers le commit
git ls-remote --tags https://github.com/actions/checkout refs/tags/v5 'refs/tags/v5^{ }'

# retrouver la version d'un SHA déjà épinglé
git ls-remote --tags https://github.com/actions/checkout | grep <8-premiers-caractères>
```

`ratchet pin` / `ratchet update` automatisent tout ça. Attention : `ratchet` ne traite que les lignes encore **taguées** — un SHA nu sans commentaire lui est illisible, comme à toi.

Construire une image dans la CI

```
image:
  needs: build                                # sans ça, les jobs partent en PARALLÈLE
  steps:
    - uses: actions/checkout@<sha>
    - uses: docker/setup-buildx-action@<sha>    # active BuildKit
    - uses: docker/build-push-action@<sha>
    with:
      context: .
      push: false
      load: true                                # sans lui, l'image reste invisible du démon
      cache-from: type=gha
      cache-to: type=gha,mode=max # mode=max garde les couches du BUILDER
      tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
```

`mode=max` est décisif : par défaut, seules les couches de l'image finale sont mises en cache — or l'étape coûteuse est dans le builder.

Taguer avec `${{ github.sha }}` : l'image dit de quel code elle sort. Un tag figé comme `0.0.1` serait écrasé silencieusement au build suivant.

Le budget temps

Chaque outil ajouté coûte du temps. **Une CI qui dépasse dix minutes cesse d'être lue** : les gens poussent et partent, et l'échec est découvert trois commits plus tard. C'est un arbitrage permanent, pas un détail.

Fiche 7 · Gitleaks — détection de secrets

Quand : avant le premier push sur un dépôt public. Le délai entre un push contenant une clé et sa première utilisation par un tiers se compte en **secondes**. Un secret poussé est un secret compromis : la seule réponse est la rotation.

Comment il décide

- **Règles** — ~170 motifs connus (`AKIA...` = clé AWS, `ghp_...` = jeton GitHub, `-----BEGIN ... PRIVATE KEY-----`). Précis, quasi aucun faux positif.
- **Entropie** — mesure du désordre d'une chaîne. Attrape les secrets maison, et produit l'essentiel des faux positifs : un hash de test, un UUID ou du base64 ont le même profil.

La recette, en 5 étapes

1 · Le job

```
secret:
  runs-on: ubuntu-22.04
  permissions:
    contents: read
    pull-requests: write          # seulement pour les commentaires de PR
  steps:
    - uses: actions/checkout@<sha>
      with:
        fetch-depth: 0           # ← SANS ÇA, IL NE SCANNE RIEN
        persist-credentials: false
    - uses: gitleaks/gitleaks-action@<sha>
      env:
        GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
        GITLEAKS_VERSION: 8.24.3
```

Deux lignes portent tout : `fetch-depth: 0` (le défaut ne récupère qu'un commit — donc aucun historique à analyser, rapport vert et vide de sens) et `GITHUB_TOKEN` (l'action interroge l'API pour connaître les commits de la PR ; sans lui elle refuse de scanner).

2 · Prouver qu'il fonctionne

Une branche jetable, un faux secret, un push, une PR. Le job doit être rouge **et nommer la règle, le fichier, la ligne**. Ne jamais tester avec une vraie clé, même révoquée.

3 · Supprimer la branche de test. Ne jamais la fusionner : le secret entrerait dans l'historique.

4 · Fusionner le workflow seul.

5 · Les exceptions — `.gitleaksignore`, une empreinte par détection :

```
<commit>:<fichier>:<règle>:<ligne>
```

Très précise : elle ne fait taire que cette détection-là. C'est aussi **le moyen le plus simple d'étouffer une vraie fuite**. *Règle d'équipe : toute ligne ajoutée doit être justifiée dans le message de commit.*

Le périmètre

	Coût	Angle mort
Tout l'historique	croît avec le dépôt	aucun
Fichiers actuels	constant	un secret committé puis supprimé
Commits du push	faible	ce qui précédait la mise en place
Audit complet + incrément	optimal	demande un fichier <i>baseline</i>

Sur un jeune dépôt : tout l'historique. Sur un dépôt de dix ans : audit une fois, baseline, puis incrément.

Vérifier avant le premier commit d'un projet

```
git add -A
git diff --cached --name-only | grep -x '.env' && echo "STOP" || echo "OK"
git grep -nEi 'password|secret|token' -- ':!*.md' | grep -v '\${'
```

Le second remontera des placeholders (`${DB_PASSWORD}`) et le code de tes outils. C'est normal : ce sont des **noms de variables**, pas des valeurs.

Deux scanners peuvent se contredire

C'est normal — règles, seuils et périmètres diffèrent. « **Rien détecté** » n'est jamais une **preuve d'absence**. C'est pour ça qu'un pipeline empile plusieurs contrôles : pas par redondance, mais parce qu'ils ont des angles morts différents.

Annexe · Commandes de référence

Docker

```
# ce qui entre réellement dans le contexte de build
docker build --no-cache --progress=plain -f - . <<'EOF'
FROM alpine
COPY . /ctx
RUN find /ctx | sort
EOF

# vérifier l'utilisateur du conteneur
docker run --rm --entrypoint id <image>          # doit afficher uid#0

# prouver qu'une couche ne s'efface pas
docker save <image> -o img.tar && mkdir x && tar xf img.tar -C x
find x -name '*.tar' -exec tar tf {} \; | grep -i <fichier>
```

Testcontainers

```
# activer la réutilisation (poste de dev uniquement)
echo "testcontainers.reuse.enable=true" >> ~/.testcontainers.properties

# le conteneur doit survivre à la fin du build
docker ps | grep postgres

# repartir de zéro après modification d'une migration
docker rm -f $(docker ps -aq --filter "label=org.testcontainers.reuse.enable=true")
```

GitHub Actions

```
gh run list --limit 3
gh run view --log-failed
actionlint          # valide le SCHÉMA, pas seulement le YAML
```

`actionlint` attrape ce qu'un parseur YAML laisse passer : un job écrit comme une liste au lieu d'un dictionnaire est du YAML parfaitement valide.

Git

```
# retrouver un contenu perdu dans une branche supprimée
git reflog --all
git log -S'motif' --all
git checkout <commit> -- <chemin>      # récupère UN fichier, sans les commits

# vérifier qu'un fichier n'est jamais entré dans l'historique
git log --all --full-history -- .env
```

Fin des fiches — parties 1 à 7.